

Semana 2: Empezamos con el Hardening de Infraestructura

Ya que decidiste quedarte en el camino difícil (BlackArch), vamos a demostrar por qué vale la pena. Hoy entramos al mundo de los **Firewalls**.

El objetivo de hoy: Aprender a configurar un muro de fuego que no solo bloquee puertos, sino que detecte comportamientos sospechosos.

Concepto Clave: iptables vs. nftables

En Linux, el firewall "tradicional" es iptables, pero el estándar moderno y más potente es nftables. Como estamos en BlackArch, vamos a aprender el moderno.

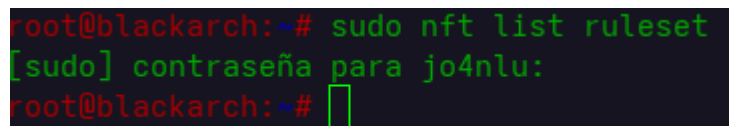
Práctica 1: Ver el estado actual de tus muros

Antes de construir, hay que ver qué hay.

Bash

Ver si hay reglas activas en nftables

sudo nft list ruleset



```
root@blackarch:~# sudo nft list ruleset
[sudo] contraseña para jo4nlu:
root@blackarch:~#
```

 **Foto 1 (Semana 2):** Seguramente saldrá vacío. Captúralo y pon en tu Word: "Sistema sin reglas activas, totalmente expuesto en todas las capas".

Práctica 2: El "Kill-Switch" de emergencia

Imagina que detectas un ataque masivo y necesitas desconectar el tráfico de red pero mantener el sistema encendido.

Bash

Instalar UFW (Uncomplicated Firewall) para una gestión rápida

sudo pacman -S ufw --needed

Bloquear TODO el tráfico entrante por defecto

sudo ufw default deny incoming

Permitir TODO el tráfico saliente (para que tú puedas navegar)

`sudo ufw default allow outgoing`

```
root@blackarch:~# sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
root@blackarch:~# sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
root@blackarch:~# █
```

 **Foto 2:** Captura estos comandos. Esto es la base del **Hardening**: "Cerrar todo por defecto y abrir solo lo necesario".

► Reto de inicio de semana

Ya configuramos que nadie puede entrar. Pero ahora tú quieres administrar tu BlackArch desde otra PC por **SSH (Puerto 22)**.

1. ¿Qué comando usarías en ufw para abrir específicamente esa "puerta" sin comprometer el resto del sistema?
2. ¿Por qué es peligroso dejar el puerto 22 abierto a todo el mundo (IP 0.0.0.0/0)?

Hardening de SSH: Cerrando la muralla

1. La sintaxis correcta de UFW

En ufw, no usamos "open", usamos **allow** (permitir). Para abrir el puerto 22 de la forma más básica:

Bash

Opción A: Por nombre de servicio

```
sudo ufw allow ssh
```

Opción B: Por número de puerto y protocolo (Más preciso)

```
sudo ufw allow 22/tcp
```

```
root@blackarch:~# sudo ufw allow ssh
[sudo] contraseña para jo4nlu:
Rules updated
Rules updated (v6)
root@blackarch:~# sudo ufw allow 22/tcp
Rules updated
Rules updated (v6)
root@blackarch:~#
```

2. El peligro de "Cualquier IP" (0.0.0.0/0)

Como bien dijiste, dejarlo abierto es invitar a Hydra a la fiesta. La regla de oro del Hardening es **limitar quién puede tocar la puerta**.

Si sabes que solo tú te conectarás desde una IP específica (ejemplo: la IP de tu casa 200.1.2.3), el comando profesional sería:

Bash

Solo permite el puerto 22 si la conexión viene de MI IP

```
sudo ufw allow from 200.1.2.3 to any port 22
```

```
root@blackarch:~# sudo ufw allow from 172.20.10.0 to any port 22
[sudo] contraseña para jo4nlu:
Rules updated
root@blackarch:~#
```

 **Foto 3 (Semana 2):** Captura la salida de `sudo ufw status verbose`. Resalta la diferencia entre una regla que dice Anywhere y una que tiene una IP específica. ¡Esa es la diferencia entre un novato y un Pro!

Nivel Experto: Protegiéndonos de Hydra y Nmap

Ya que mencionaste a Hydra, vamos a configurar una defensa activa en tu BlackArch para que, si alguien intenta atacarte, el firewall lo banee automáticamente.

A. Limitar intentos (ufw limit)

UFW tiene una función "mágica" llamada limit. Si una IP intenta conectarse 6 veces en menos de 30 segundos, el firewall la bloquea automáticamente. Es la kriptonita de los scripts básicos de fuerza bruta.

Bash

Borramos la regla anterior

```
sudo ufw delete allow 22/tcp
```

Aplicamos la regla con límite de velocidad

```
sudo ufw limit 22/tcp
```

```
root@blackarch:~# sudo ufw delete allow 22/tcp
Rules updated
Rules updated (v6)
root@blackarch:~# sudo ufw limit 22/tcp
Rules updated
Rules updated (v6)
root@blackarch:~#
```

B. Engañando al atacante (Cambio de puerto)

Si quieres que Hydra ni siquiera encuentre tu SSH, cambia el puerto. En lugar del 22, usa uno alto como el 2222.

Resumen para tu Word (Hardening de SSH)

- **Riesgo:** Ataques de fuerza bruta (Hydra) y escaneo de servicios (Nmap).
- **Defensa Nivel 1:** Cambiar el puerto default (del 22 a uno aleatorio).
- **Defensa Nivel 2:** Usar ufw limit para bloquear ráfagas de intentos.
- **Defensa Nivel 3 (La mejor):** Permitir acceso solo desde IPs conocidas (allow from [IP]).

► Reto de Seguridad #10

Ya tenemos el firewall encendido y el SSH protegido. Pero hay un "chismoso" en la red: el **Ping**. Por defecto, tu BlackArch responde a los pings, lo que le confirma a un atacante que tu máquina está encendida y lista para ser escaneada.

1. **¿Es posible configurar el firewall para que tu máquina sea "invisible" (que no responda al ping) pero que tú sí puedas navegar por internet normalmente?**
2. **¿Qué protocolo (del que hablamos ayer) tendrías que filtrar en el firewall para lograr este "Modo Invisible"?**

Modo Mentor: Ajustando el Radar

1. **¿Protocolo TCP? (Corrección):** ¡Negativo, bro! Recuerda lo que vimos ayer: el **Ping no tiene puertos**, por lo tanto, no es TCP ni UDP. El protocolo que debes filtrar es el **ICMP** (Internet Control Message Protocol). Si bloqueas TCP pensando que bloqueas el ping, tu máquina seguirá gritando "¡Aquí estoy!" a cualquiera que le envíe un paquete ICMP Echo Request.
2. **Sobre cambiar la IP:** Es una opción, pero en una red profesional las IPs suelen ser fijas o asignadas por DHCP. Es mucho más elegante y profesional ser un **"Agujero Negro" (Drop)**: que tu IP sea la correcta, pero que el atacante no reciba *absolutamente nada* de vuelta, haciéndole creer que la dirección está vacía.

Práctica en BlackArch: Activando el "Modo Fantasma"

UFW es "amigable", pero para bloquear el ping a nivel experto tenemos que entrar en sus archivos de configuración internos. Vamos a editar las reglas que se ejecutan *antes* que cualquier otra.

1. Editando las reglas de ICMP

Sigue estos pasos y captura el proceso para tu Word:

Bash

Abrimos el archivo de reglas "primarias" de UFW

```
sudo nano /etc/ufw/before.rules
```

Busca las líneas que dicen `ok icmp codes` (están cerca del principio). Verás algo como esto: `-A ufw-before-input -p icmp --icmp-type echo-request -j ACCEPT`

El Cambio: Cambia ese `ACCEPT` por un **DROP**.

Diferencia técnica para tu Word:

- **REJECT:** Le dice al atacante "No quiero hablar contigo". (Él sabe que estás ahí porque le respondiste que no).
- **DROP:** Simplemente ignora el paquete. El atacante se queda esperando hasta que le da "Timeout". **Este es el verdadero modo invisible.**

2. Reiniciar el muro

Bash

```
sudo ufw reload
```

3. Prueba de fuego (Auto-auditoría)

Desde otra máquina (o pidiéndole a un colega), intenta hacerte ping.

Bash

ping [Tu_IP_de_BlackArch]

```
root@blackarch:~# ping 170.20.10.5
PING 170.20.10.5 (170.20.10.5) 56(84) bytes de datos.
^C
--- 170.20.10.5 estadísticas ping ---
25 paquetes transmitidos, 0 recibidos, 100% packet loss, time 24608ms
root@blackarch:~#
```

 **Foto 4 (Semana 2):** Captura la pantalla donde se vea que el ping no responde ("Request timeout"). Pon en tu Word: "Configuración de Hardening ICMP: La máquina ya no responde a Echo Requests, dificultando el descubrimiento de activos mediante escaneo básico".

Resumen para tu Word (Modo Invisible)

- **Protocolo a filtrar:** ICMP (Capa 3).
- **Acción recomendada:** DROP (en lugar de REJECT) para no dar pistas al atacante.
- **Resultado:** Nmap informará que el host está "Down" (apagado), a menos que use técnicas de escaneo más avanzadas que veremos luego.

Reto de Seguridad #11: El Escaneo Silencioso

Ya no respondes al Ping. El atacante, frustrado, decide usar **Nmap** con un escaneo tipo **SYN Stealth Scan** (-sS). Este escaneo no completa la conexión (Capa 4), por lo que algunos firewalls viejos no lo ven.

1. Si el atacante usa -sS, ¿tu firewall UFW lo detectará y bloqueará por defecto o necesitamos algo más "inteligente"?
2. ¿Qué herramienta de BlackArch (que instalamos ayer) sirve para "mirar" en tiempo real si alguien nos está escaneando los puertos?

Hardening Avanzado: Detectar al "Chismoso"

UFW por defecto es silencioso. Si alguien te escanea, no te enteras. Vamos a activar los **Logs** (registros) para que el sistema nos avise.

1. Activar logs de Firewall

Bash

sudo ufw logging on

sudo ufw logging medium

```
root@blackarch:~# sudo ufw logging on
[sudo] contraseña para jo4nlu:
Logging enabled
root@blackarch:~# sudo uwf logging medium
sudo: uwf: command not found
root@blackarch:~# sudo ufw logging medium
Logging enabled
root@blackarch:~#
```

2. Leer los logs en tiempo real

Aquí es donde BlackArch se pone interesante. Vamos a vigilar el archivo donde el kernel anota los bloqueos:

Bash

Ver las últimas líneas del log y seguir esperando nuevas

 **Foto 6:** Aquí verás exactamente qué IP intentó entrar y a qué puerto. Si ves una IP intentando entrar a 10 puertos distintos en un segundo, **tienes un escaneo de Nmap en la cara.**

Resumen para tu Word (Detección de Escaneos)

- **Herramienta de monitoreo:** tcpdump (Capa 3 y 4).
- **Estrategia de evasión:** Configurar el Firewall en modo DROP para que Nmap se quede esperando y tarde una eternidad en terminar el escaneo.

- **Detección:** Revisar /var/log/ufw.log para identificar patrones de escaneo (muchos intentos en poco tiempo desde una misma IP).

► Reto de Seguridad #12 (Final de Firewalls)

Imagina que el atacante es persistente. Ya viste su IP en los logs (192.168.1.50) y te está molestando con escaneos constantes.

1. **¿Cuál es el comando de ufw para banear (bloquear) COMPLETAMENTE a esa IP específica y que no pueda ni siquiera intentar tocar tu firewall?**
2. **Si bloqueas a ese atacante en el Firewall, ¿él podrá seguir viéndote con un arping (Capa 2)?** (Piensa bien esta, recuerda lo que hablamos ayer del "pegamento" de la red).

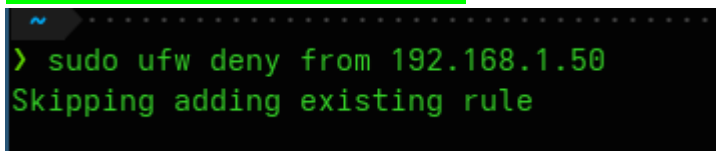
🔧 1. El comando de baneo total

Para bloquear a la IP 192.168.1.50 y que no pueda ni "respirar" en tu dirección, el comando completo en BlackArch es:

Bash

Bloquea todo el tráfico proveniente de esa IP

```
sudo ufw deny from 192.168.1.50
```



```
> sudo ufw deny from 192.168.1.50
Skipping adding existing rule
```

📷 **Foto 7 (Semana 2):** Ejecuta `sudo ufw status`. Verás una línea que dice `Anywhere FROM 192.168.1.50 DENY`. En tu Word pon: "Baneo preventivo de host hostil detectado en los logs".

2. La trampa de la Capa 2 (ARP vs Firewall)

¿Por qué te sigue viendo con arping? Porque el Firewall (ufw/iptables) vive en la **Capa 3 (Red)** y **Capa 4 (Transporte)**. El protocolo **ARP** vive en la **Capa 2 (Enlace)**.

Cuando el atacante lanza un arping, le está preguntando al switch: "¿Quién tiene esta IP?". Tu tarjeta de red responde automáticamente con su dirección MAC **antes** de que el paquete llegue siquiera al Firewall. Para el firewall, el paquete todavía no es "tráfico", es solo el pegamento de la red funcionando.

💡 **Nota de Hardening:** Si quieres ser invisible incluso al arping, tendrías que usar herramientas de nivel más bajo como arptables o configurar el kernel para que no responda a peticiones ARP, pero eso suele romper la red si no sabes lo que haces.

Siguiente nivel: Análisis de Vulnerabilidades con Nmap

Ya sabemos defendernos. Ahora vamos a ponernos el sombrero de **Auditor (Red Team)**. No solo queremos saber si un puerto está abierto, queremos saber **qué tan vulnerable es**.

Nmap Scripting Engine (NSE)

Nmap no solo escanea puertos; tiene miles de scripts (.nse) que pueden detectar vulnerabilidades específicas, como si un servidor tiene contraseñas por defecto o si es vulnerable a ataques famosos.

Práctica: Escaneo de "Banner Grabbing"

Vamos a obligar a los servicios a decirnos quiénes son y qué versión usan.

Bash

-sV: Determina la versión del servicio

--script=banner: Intenta capturar el mensaje de bienvenida del servicio

```
nmap -sV --script=banner [IP_Objeto]
```

```
sudo nmap -p- -sS -sV -sC -A -T4 192.168.1.1
[sudo] contraseña para jo4nlu:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-04 22:01 -0500
Nmap scan report for 192.168.1.1
Host is up (0.0032s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
23/tcp    open  telnet       BusyBox telnetd 1.00-pre7 - 1.14.0
banner-strings:
  GenericLines:
    F6600P
    Login:
    Password:
    Access denied
    Login:
  GetRequest:
    HTTP/1.0
    F6600P
    Login: Password:
    Access denied
    Login:
  Help:
    HELP
```

📸 **Foto 8:** Si el objetivo tiene SSH, verás algo como SSH-2.0-OpenSSH_8.2. En tu Word explica: "El Banner Grabbing permite al atacante saber la versión exacta del software para buscar un exploit específico en bases de datos como Exploit-DB".

▶ Reto de Auditoría #13

Estás auditando un servidor web y lanzas un escaneo de Nmap. El resultado te dice que el puerto **80 (HTTP)** está abierto y que el servidor es un **Apache 2.4.49**.

1. ¿Por qué es más peligroso que Nmap te diga la versión exacta (2.4.49) a que solo te diga que el puerto está "Open"?
2. ¿Qué comando de Nmap usarías para buscar vulnerabilidades conocidas (exploits) automáticamente en ese servicio? (Pista: El script se llama vuln).

Ajuste Técnico: El comando Nmap

Para la segunda pregunta, te acercaste, pero faltó la "palabra mágica".

- -sS hace el escaneo silencioso.
- -sV saca la versión.
- **La corrección:** Para usar los scripts automatizados de vulnerabilidades de Nmap, necesitas el parámetro --script.

Práctica en BlackArch: Búsqueda de Vulnerabilidades

Para tu Word, vamos a registrar cómo se hace una auditoría de vulnerabilidades automatizada y manual.

1. Escaneo Automatizado (El Nmap Destructor)

Este comando le dice a Nmap: *"Busca puertos, dime la versión, y luego corre TODOS los scripts clasificados como 'vulnerabilidad' contra lo que encuentres"*.

Bash

Advertencia: Este escaneo es muy ruidoso y puede tumbar servicios inestables

nmap -sV --script vuln [IP_Objeto]

```
> sudo nmap -sV --script vuln 192.168.1.1
[sudo] contraseña para jo4nlu:
Starting Nmap 7.99 ( https://nmap.org ) at 2026-05-04 23:08 -0500
Nmap scan report for 192.168.1.1
Host is up (0.0054s latency).
Not shown: 996 closed tcp ports (reset)
PORT      STATE SERVICE  VERSION
23/tcp    open  telnet   BusyBox telnetd 1.00-pre7 - 1.14.0
| fingerprint-strings:
|   GenericLines:
|     F6600P
|     Login:
|     Password:
|     Access denied
|     Login:
|   GetRequest:
|     HTTP/1.0
|     55555555
```

 **Foto 9 (Semana 2):** Captura la salida. Si Nmap encuentra algo, verás referencias que empiezan con **CVE-XXXX-XXXX**. En tu Word explica: "El flag --

script vuln automatiza la detección de fallos conocidos basándose en las versiones de los servicios descubiertos".

2. Búsqueda Manual Offline (Searchsploit)

A veces, no quieres hacer ruido en la red del cliente lanzando scripts de vulnerabilidad. Simplemente anotas la versión (ej. Apache 2.4.49) y buscas en tu propia base de datos offline.

En BlackArch, usamos la herramienta searchsploit (que lee la base de datos de Exploit-DB sin necesidad de internet).

Bash

```
# Instalamos la base de datos si no la tienes
```

```
sudo pacman -S exploitdb --needed
```

Buscamos exploits específicos para esa versión

searchsploit apache 2.4.49

Exploit Title	Path
Apache + PHP < 5.3.12 / < 5.4.2 - cgi-	php/remote/29290.c
Apache + PHP < 5.3.12 / < 5.4.2 - Remo	php/remote/29316.py
Apache CXF < 2.5.10/2.6.7/2.7.4 - Deni	multiple/dos/26710.txt
Apache HTTP Server 2.4.49 - Path Trave	multiple/webapps/50383.sh
Apache mod_ssl < 2.8.7 OpenSSL - 'Open	unix/remote/21671.c
Apache mod_ssl < 2.8.7 OpenSSL - 'Open	unix/remote/47080.c
Apache mod_ssl < 2.8.7 OpenSSL - 'Open	unix/remote/764.c
Apache OpenMeetings 1.9.x < 3.1.0 - '	linux/webapps/39642.txt
Apache Tomcat < 5.5.17 - Remote Direct	multiple/remote/2061.txt
Apache Tomcat < 6.0.18 - 'utf8' Direct	multiple/remote/6229.txt
Apache Tomcat < 6.0.18 - 'utf8' Direct	unix/remote/14489.c
Apache Tomcat < 9.0.1 (Beta) / < 8.5.2	jsp/webapps/42966.py
Apache Tomcat < 9.0.1 (Beta) / < 8.5.2	windows/webapps/42953.txt
Apache Xerces-C XML Parser < 3.1.2 - D	linux/dos/36906.txt
Webfroot Shoutbox < 2.32 (Apache) - Lo	linux/remote/34.pl

 **Foto 10:** Verás una lista de exploits listos para usarse (código en Python, Ruby, C, etc.). En tu Word pon: "Searchsploit permite buscar vectores de ataque (exploits) de forma silenciosa utilizando la información recopilada en la fase de reconocimiento".

Resumen para tu Word (Auditoría y Vulnerabilidades)

- **El peligro del Banner Grabbing:** Exponer las versiones exactas del software (ej. Apache 2.4.49) permite a los atacantes buscar exploits específicos (CVEs) sin esfuerzo.
- **Automatización (Nmap NSE):** `nmap --script vuln` cruza automáticamente los servicios detectados con bases de datos de vulnerabilidades.
- **Búsqueda Silenciosa (Searchsploit):** Herramienta esencial para consultar la base de datos de Exploit-DB localmente y planificar el ataque sin alertar a los firewalls.

Has dominado la fase de **Reconocimiento** (escanear puertos y servicios) y la de **Escaneo de Vulnerabilidades** (encontrar el agujero). Siguiendo la lógica de un ataque real (la *Kill Chain*), ¿cuál crees que es la siguiente fase natural que un Red Teamer debe ejecutar una vez que ya tiene el exploit (el código de ataque) en sus manos?

La Fase: Post-Exploitation

Una vez que logras entrar al sistema (aunque sea con un usuario sin privilegios), empieza la verdadera diversión. La post-explotación no es solo "mirar", es **tomar el control total**.

Se divide principalmente en estas cuatro acciones:

1. **Enumeración Interna:** Recolectar información desde dentro. ¿Qué archivos hay? ¿Qué otros usuarios están conectados? ¿Hay contraseñas guardadas en el historial?
2. **Escalada de Privilegios (Privilege Escalation):** Eres un usuario normal, pero quieres ser **Root** (Linux) o **SYSTEM** (Windows). Buscas fallos en el kernel o servicios mal configurados para "subir de nivel".
3. **Persistencia:** Quieres asegurarte de que, si el dueño reinicia la computadora, tú no pierdas el acceso. Creas una "puerta trasera" (backdoor).
4. **Movimiento Lateral / Pivoting:** Usar esa computadora como puente para atacar a otras máquinas de la misma red que no eran visibles desde internet.

Herramientas de Post-Ex en BlackArch

Para tu Word, registra estas herramientas que se usan cuando ya estás "dentro" pero quieres ser el dueño total:

Herramienta	Función en Post-Explotación
LinPEAS / WinPEAS	Scripts automáticos que buscan CUALQUIER fallo para escalar privilegios.
Mimikatz	(Para Windows) Extrae contraseñas y hashes directamente de la memoria RAM.
Powersploit	Colección de scripts de PowerShell para dominar entornos Windows.
Meterpreter	La "shell" avanzada de Metasploit que tiene comandos de post-ex integrados.

Ejemplo para tu Word: El comando "Post-Ex" por excelencia

Si estás usando Metasploit y lograste una sesión de **Meterpreter**, el primer comando de recolección de información que debes poner en tu reporte es:

Bash

Dentro de la sesión de Meterpreter:

getuid # ¿Quién soy?

sysinfo # ¿En qué sistema estoy?

hashdump # Intentar robar la base de datos de contraseñas (hashes)

 **Foto 11 (Semana 2):** Si no tienes una sesión activa, captura el menú de ayuda de search post en Metasploit. Pon en tu Word: "Metasploit tiene módulos específicos de 'Post' para automatizar la recolección de evidencia una vez vulnerado el sistema".

Reflexión de Mentor: "Parchado no significa seguro"

Me comentaste que el dispositivo está parchado. En el mundo real, eso significa que el atacante deja de buscar **Exploits de Software** y empieza a buscar **Exploits de Configuración**.

- *Ejemplo:* El sistema está actualizado, pero el administrador dejó una carpeta con permisos 777 donde guardó una copia de la llave privada de SSH. **¡Eso es Post-Explotación pura!**

► Reto de Post-Explotación #14

Imagina que entraste a un servidor Linux como el usuario invitado. No puedes leer la carpeta /root porque no tienes permisos.

1. **¿Cómo se llama la acción de intentar pasar de invitado a root?**
2. **Si encuentras un archivo llamado config.php con las credenciales de la base de datos, ¿en qué sub-fase de la Post-Explotación estarías?**

¿Qué te parece si para la siguiente clase nos enfocamos en cómo buscar esos "fallos de configuración" que los parches no pueden arreglar?

🔧 Ajuste Técnico de Post-Explotación

1. De Invitado a Root: Escalada de Privilegios Vertical

Mencionaste "privilegios, sudo, chmod". ¡Exacto! Pasar de un usuario sin permisos (invitado) al administrador supremo (root) se conoce técnicamente como **Escalada de Privilegios Vertical** (Privilege Escalation).

- Usas comandos como sudo -l para ver qué privilegios mal configurados tienes, o explotas archivos con permisos mal asignados (chmod).

2. Encontrar el config.php: Enumeración y Saqueo (Looting)

Dijiste que es "parte del objetivo" y tienes toda la razón. En ciberseguridad, a esta sub-fase se le llama **Looting** (Saqueo o Recolección de Botín) dentro de la **Enumeración Interna**.

- Encontrar contraseñas en texto plano dentro de un archivo de configuración no es "upload" (eso sería subir un archivo), sino extraer el valor que te va a permitir saltar a la Base de Datos o comprometer otros servicios.

🔗 Práctica en BlackArch: Buscando el "Botín" de Configuración

Para cerrar con broche de oro esta sección en tu reporte de Word, vamos a ver el comando que usarías estando dentro de la máquina víctima para encontrar ese config.php de forma instantánea:

Bash

```
# Buscar archivos que se llamen 'config' y contengan extensiones web (.php, .xml, .json)
# desde el directorio actual hacia abajo
find . -type f -name "*config*.php" 2>/dev/null
```

Buscar la palabra "password" o "passwd" dentro de archivos de texto de forma recursiva

```
grep -ri "password" *.php 2>/dev/null
```

 **Foto 12 (Final de la Clase):** Captura estos comandos de búsqueda en tu terminal. En tu Word, añade esta nota de cierre: "La post-explotación efectiva depende de una enumeración minuciosa. Un solo archivo de configuración olvidado (config.php) con credenciales por defecto puede comprometer la infraestructura entera sin necesidad de explotar vulnerabilidades del kernel".

Resumen Ejecutivo para tu Word (Clase 1.3)

- **Post-Explotación:** Fase posterior al acceso inicial donde se recolecta información interna, se mantienen accesos y se escalan privilegios.
- **Escalada de Privilegios:** El proceso de abusar de malas configuraciones (sudo, tareas cron, permisos de archivos) para transformarse en el usuario root.
- **Looting (Botín):** Extracción de secretos del sistema (llaves SSH, archivos de configuración, hashes de contraseñas) para expandir el alcance del ataque.

